

Fine-Tuning Large Language Models for Natural Language to SQL: A Comparative Study of Qwen 2.5-Coder and LLaMA 3.1 Using QLoRA

Muhammad Danish¹, Ali bin Salman¹, and Shahzaib Rauf¹

Department of Computer Science, FAST NUCES, Islamabad, Pakistan
{i221305, i220894, i228220}@nu.edu.pk

Abstract. Converting natural language queries into executable SQL remains one of the most practically impactful challenges in the field of natural language processing. While modern large language models (LLMs) possess strong general reasoning abilities, they routinely struggle with the rigid syntactic demands of SQL, leading to schema hallucination and incorrect join resolution. This paper investigates whether domain-specialized parameter-efficient fine-tuning can close this gap on consumer-grade hardware. We apply QLoRA (Quantized Low-Rank Adaptation) with 4-bit NormalFloat quantization to two architecturally distinct open-source LLMs—Qwen 2.5-Coder-7B-Instruct and Meta-LLaMA 3.1-8B-Instruct—trained on a unified corpus built from the Spider, SPaRc, and CoSQL benchmarks. Both models are evaluated on the complete Spider development set (1,034 examples) under identical conditions using Strict Execution Accuracy and Relaxed Set-Match Accuracy. Qwen 2.5-Coder achieves 69.3% and 72.5% on these metrics, while LLaMA 3.1 reaches 67.8% and 70.3%, with both exceeding their respective zero-shot baselines by more than 21 percentage points. We analyze the architectural factors underlying these differences, document practical training challenges and their resolutions, and provide an ablation study examining context-window size effects. Our pipeline and findings offer a reproducible reference for resource-constrained NL2SQL adaptation.

Keywords: NL2SQL · Text-to-SQL · QLoRA · LLaMA 3.1 · Qwen 2.5-Coder · Instruction Tuning · Spider Benchmark · Parameter-Efficient Fine-Tuning

1 Introduction

Relational databases underpin virtually every modern organization, yet the Structured Query Language required to interrogate them remains a significant barrier for non-technical users. Natural Language to SQL (NL2SQL) systems seek to remove this barrier by translating plain-language questions directly into executable queries [1].

Contemporary LLMs have dramatically raised the ceiling on zero-shot NL2SQL performance, yet three persistent failure modes limit their practical utility [6].

Schema hallucination arises when a model generates syntactically plausible but schema-invalid column or table names. *Redundant join insertion* occurs when the model unnecessarily joins auxiliary tables whose columns are never referenced in the output. *Context amnesia* affects multi-turn interactions: filter conditions established in one dialogue turn are silently discarded in the next. These issues motivate supervised fine-tuning on curated, schema-aware corpora.

Our work makes the following contributions:

1. **Dual model fine-tuning and evaluation.** We train and evaluate QLoRA-adapted versions of Qwen 2.5-Coder-7B-Instruct and Meta-LLaMA 3.1-8B-Instruct on a common NL2SQL curriculum, enabling a controlled architectural comparison.
2. **Unified multi-source dataset pipeline.** A preprocessing pipeline merges Spider [2], SPaC [3], and CoSQL [4] into a single instruction-tuning format, incorporating SQL validation, schema normalization, and a cumulative-history transformation for multi-turn examples.
3. **Systematic benchmarking.** Both fine-tuned models are assessed on all 1,034 examples of the Spider development set under Strict Execution Accuracy (EM) and Relaxed Set-Match Accuracy (EX), accompanied by structured failure analysis.
4. **Ablation study.** We examine how context-window length and batch size interact with VRAM constraints on Tesla T4 hardware, providing practical guidance for practitioners.
5. **Infrastructure reproducibility.** We document and resolve five non-trivial training obstacles—multiprocessing deadlocks, VRAM gradient offloading, compiler incompatibilities, tokenizer operator fragmentation, and a TRL batch-normalization bug specific to LLaMA 3.1—rarely discussed in benchmark papers.

The remainder of this paper proceeds as follows. Section 2 surveys related work. Section 3 describes both model architectures and the shared training methodology. Section 4 presents experimental results and comparative analysis. Section 5 details the ablation study. Section 6 discusses findings, limitations, and future directions. Section 7 concludes the paper.

2 Related Work

2.1 Benchmark Datasets for NL2SQL

The trajectory of NL2SQL research has been driven by progressively demanding evaluation benchmarks. The WikiSQL corpus introduced by Zhong et al. [1] enabled the first large-scale study of query generation from natural language, using a reinforcement learning objective with execution rewards. Despite its scale, WikiSQL operates exclusively on single-table schemas and templated query patterns, which do not capture the JOIN-heavy complexity of real enterprise databases.

Yu et al. [2] addressed this limitation with Spider, a cross-domain benchmark comprising 10,181 question–query pairs spanning 200 heterogeneous databases. Spider demands generalization to previously unseen schemas, handling of nested subqueries, multi-table joins, and set operations—making it the canonical NL2SQL evaluation testbed. The same group extended Spider in two directions: SParC [3] introduced sequential multi-turn dependencies where each question builds on prior context, while CoSQL [4] added interactive state-tracking, simulating a database explorer in which user intent may be ambiguous or require clarification.

Our training corpus deliberately combines all three Spider-lineage datasets (excluding WikiSQL) to expose both models to the full range of SQL complexity and conversational structure.

2.2 Specialized Architectures

Prior to the LLM era, leading approaches built task-specific inductive biases directly into the model architecture. RAT-SQL [9] encoded the database schema as a directed graph and propagated relation-aware attention weights—primary-key, foreign-key, and column–table membership edges—through the transformer’s self-attention layers, yielding state-of-the-art results on Spider. IGSQ [10] incorporated interaction-level graph encoding for contextual multi-turn queries, while BRIDGE [11] injected literal database values alongside schema tokens to improve value-conditioned filtering. These architectures required training from scratch and could not exploit the broad semantic priors of pre-trained language models.

2.3 LLMs and Instruction Tuning

The shift to pre-trained LLMs reframed NL2SQL as a prompting and fine-tuning problem rather than an architectural design challenge. InstructGPT [5] demonstrated that RLHF-based alignment substantially improves zero-shot task compliance. Gao et al. [6] conducted a large-scale benchmark evaluation revealing that while GPT-4-class models handle simple single-table queries well, fine-tuned specialized models retain a meaningful advantage on schema-complex queries requiring strict identifier adherence. MAC-SQL [7] decomposed SQL generation into distinct reasoning, linking, and execution phases orchestrated by cooperating agents. ROUTE [8] applied resilient multitask tuning across heterogeneous SQL tasks to improve cross-domain generalization.

2.4 Parameter-Efficient Fine-Tuning

Full fine-tuning of multi-billion-parameter models requires hundreds of gigabytes of GPU memory for gradients and optimizer states, placing it beyond reach for most researchers. LoRA [13] circumvented this by freezing pre-trained weights and injecting low-rank trainable matrices into each transformer layer, reducing

trainable parameters by more than 99% with minimal accuracy degradation. QLoRA [12] combined LoRA with 4-bit NormalFloat (NF4) quantization of the frozen weights, enabling fine-tuning of 7–8 billion parameter models on a single 16 GB GPU. Both Qwen 2.5 [14] and LLaMA 3.1 [15] are available in Unsloth-optimized 4-bit variants [16] that further reduce kernel-level memory overhead, and these are the base checkpoints used in our study.

3 Methodology

3.1 Model Architectures

Qwen 2.5-Coder-7B-Instruct. Qwen 2.5-Coder is a 7-billion parameter model developed by Alibaba Cloud [14], pre-trained on an extensive corpus of source code and technical documentation before undergoing instruction tuning. Its architecture incorporates several properties that are particularly relevant for SQL generation.

Rotary Positional Embeddings (RoPE) provide relative position encoding that generalizes well to sequences longer than those seen during training, which is beneficial for SQL queries containing deeply nested clauses and long schema definitions.

SwiGLU Activation. The feed-forward sublayer employs the SwiGLU gating function, which improves gradient flow relative to standard ReLU activations:

$$\text{SwiGLU}(x) = \text{Swish}(xW_1) \otimes (xW_2), \quad (1)$$

where $\otimes$ denotes element-wise multiplication and $\text{Swish}(z) = z \cdot \sigma(z)$.

Code-domain pre-training provides Qwen 2.5-Coder with an ingrained familiarity with syntactic structures—parentheses, operators, reserved keywords—shared between general programming languages and SQL, offering a favorable initialization for schema-constrained generation.

Meta-LLaMA 3.1-8B-Instruct. LLaMA 3.1-8B is Meta’s 8-billion parameter general-purpose instruction model [15], trained on a broad multilingual corpus that includes a substantial but non-dominant fraction of code. Its architectural characteristics distinguish it clearly from Qwen 2.5-Coder in ways that bear on NL2SQL performance.

Grouped Query Attention (GQA). LLaMA 3.1 partitions attention heads into groups that share a single key–value head, substantially reducing the KV-cache memory footprint during inference. With 32 query heads and 8 key-value heads, this design allows larger effective batch sizes within the 16 GB VRAM budget without compromising attention quality.

Extended native context window. LLaMA 3.1 natively supports context lengths up to 128K tokens via RoPE scaling with a rope-theta of 500,000 and a high-frequency factor of 4.0, as confirmed by the model configuration observed during training. Although we restrict the context to 2,048 tokens during fine-tuning for

hardware compatibility, this architectural capacity means LLaMA 3.1 is well-positioned for deployment scenarios with long schema definitions.

Broad instruction following. LLaMA 3.1’s pre-training distribution spans diverse natural language tasks, which we hypothesize yields stronger question-comprehension abilities—beneficial when a natural language query is ambiguous or uses indirect phrasing.

Table 1 summarizes the key architectural differences between the two models.

Table 1. Architectural Comparison of the Two Base Models

Property	Qwen 2.5-Coder-7B	LLaMA 3.1-8B
Parameters	7B	8B
Pre-training focus	Code + general	General + code
Attention type	Standard MHA	Grouped Query Attention
KV heads	8	8
Hidden size	3,584	4,096
Intermediate size	18,944	14,336
Native context	32K	128K
Activation	SwiGLU	SiLU
Positional encoding	RoPE	RoPE (rope-theta 500K)
Instruction format	ChatML	LLaMA-3 header

3.2 QLoRA: Mathematical Formulation

Standard full fine-tuning of a 7–8B parameter model in FP32 requires approximately 112–128 GB of GPU memory for parameters, gradients, and Adam optimizer states—far beyond the 14.56 GB available on a Tesla T4. QLoRA addresses this through two compounding techniques.

Low-Rank Adaptation. For each weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA freezes the pre-trained weights and introduces a trainable update ΔW factored as the product of two low-rank matrices:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x, \quad (2)$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ with rank $r \ll \min(d, k)$, and α is a scalar controlling the magnitude of the adaptation. We set $r = 16$ and $\alpha = 16$ for both models, applying adapters to all seven linear projection modules: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. This yields 41,943,040 trainable parameters—approximately 0.52% of the total parameter count for LLaMA 3.1, as confirmed during training.

4-bit NormalFloat Quantization. The frozen base weights W_0 are stored in 4-bit NormalFloat (NF4) format. During each forward pass, a double-quantized dequantization recovers the full-precision values on the fly:

$$Y^{\text{BF16}} = X^{\text{BF16}} \cdot \text{doubleDequant}(c_2^{\text{FP32}}, c_1^{k\text{-bit}}, W^{\text{NF4}}) + X^{\text{BF16}} L_1 L_2, \quad (3)$$

where c_2^{FP32} and $c_1^{k\text{-bit}}$ are nested quantization constants (double-quantized scaling factors), W^{NF4} holds the frozen 4-bit weights, and L_1, L_2 are the BFloat16 LoRA adapters. For LLaMA 3.1, this reduced peak VRAM to 10.4 GB (3.4 GB attributable to the LoRA adaptation), well within the T4’s 14.56 GB budget.

3.3 Dataset Construction

Source Datasets and Filtering. Our training corpus draws from three benchmarks:

- **Spider** (`train_spider.json` + `train_others.json`): single-turn, cross-domain examples covering nested queries, aggregations, and multi-table joins across 200 databases.
- **SParC**: multi-turn sequential queries where each turn refines or extends the previous question over the same database.
- **CoSQL**: conversational queries with dialogue-state tracking, including clarification exchanges.

WikiSQL is explicitly excluded (`WIKISQL_CAP = 0`) to avoid “WikiSQL poisoning”—the tendency of its narrow single-table query distribution to corrupt performance on the more complex join-heavy queries that constitute the majority of Spider. The final corpus contains 14,104 training examples, confirmed from the notebook output `Loaded 14,104 examples. Ready for training!`

Each example undergoes validation before inclusion: queries lacking a `SELECT` statement, referencing tables absent from the parsed schema, or applying `GROUP BY` to an unqualified `SELECT *` are discarded. Algorithm 1 formalizes this procedure.

Schema Representation. Each database schema is parsed from the Spider `tables.json` format into executable `CREATE TABLE` DDL, preserving original column types (`INTEGER`, `TEXT`, `REAL`, `DATE`, `DATETIME`, `BOOLEAN`) and including explicit `PRIMARY KEY` and `FOREIGN KEY` declarations. This gives both models a structured, unambiguous representation of relational constraints. An example schema excerpt for a hospital database is shown below:

```
CREATE TABLE physician (
  employeeid TEXT PRIMARY KEY,
  name TEXT, position TEXT, ssn TEXT
);
CREATE TABLE department (
```

Algorithm 1 SQL Quality Filtering Before Dataset Construction**Require:** Raw dataset D_{raw} , schema lookup $\mathcal{S}$ **Ensure:** Filtered dataset D_{clean}

```

1:  $D_{\text{clean}} \leftarrow \emptyset$ 
2: for each  $(q, a, db\_id) \in D_{\text{raw}}$  do
3:    $s \leftarrow \mathcal{S}[db\_id]$ 
4:   if  $a.\text{upper}()$  does not contain SELECT then
5:     continue
6:   end if
7:   if any table in FROM/JOIN( $a$ )  $\notin \text{tables}(s)$  then
8:     continue
9:   end if
10:  if SELECT  $*$  and GROUP BY both appear in  $a$  then
11:    continue
12:  end if
13:   $D_{\text{clean}} \leftarrow D_{\text{clean}} \cup \{(q, a, s)\}$ 
14: end for

```

```

departmentid TEXT PRIMARY KEY,
name TEXT, head TEXT,
FOREIGN KEY (head) REFERENCES physician(employeeid)
);

```

Cumulative History Transformation. For multi-turn datasets (SPaC and CoSQL), each question in isolation is decontextualized and unlearnable. We apply a cumulative-history transformation where the input I_n for the n -th turn within a dialogue concatenates the schema with all prior exchanges:

$$I_n = \text{Schema} \oplus \bigoplus_{i=1}^{n-1} (\text{User}_i \oplus \text{Assistant}_i) \oplus \text{User}_n. \quad (4)$$

This reframes multi-turn state tracking as a reading-comprehension problem: the model resolves co-references (“those students”, “order them by GPA”) by attending to previously established filter conditions in the flattened context rather than maintaining an external state vector.

System Prompt Design. A consistent six-rule system prompt is applied at both training and inference time for both models:

1. Output only raw SQL—no markdown fences, no prose.
2. Never reference identifiers absent from the provided schema.
3. Return columns in the exact order implied by the question.
4. Avoid joining tables unless the join is strictly necessary.
5. Prefer **ORDER BY** ... **LIMIT 1** over a **MAX()** subquery for extremum retrieval.
6. Apply all **WHERE** predicates before any **ORDER BY** or **LIMIT** clause.

This prompt directly targets the three failure modes identified in the introduction. Using the same prompt for both models ensures that any accuracy difference reflects model capacity rather than prompting strategy.

Tokenization Operator Repair. Both tokenizers occasionally split multi-character comparison operators across tokens during generation (e.g., `>=` becomes `> =`). A post-generation regex repair step corrects this before SQL execution:

$$> = \rightarrow >=, \quad < = \rightarrow <=, \quad ! = \rightarrow !=. \quad (5)$$

4 Experimental Setup and Results

4.1 Hardware and Training Configuration

All experiments ran on the Kaggle platform under the following hardware configuration:

- **GPU:** NVIDIA Tesla T4, 14.56 GB VRAM (2 available; single-GPU training used for LLaMA 3.1).
- **CPU:** 4-core Intel Xeon.
- **System RAM:** 29 GB.

Table 2 presents the complete hyperparameter configuration for both models. Both share the same LoRA rank, effective batch size, learning rate, and optimizer. The primary implementation differences are the chat-template format and the response-extraction logic used during inference.

4.2 Training Dynamics

LLaMA 3.1-8B Training Curve. The LLaMA 3.1 model was trained for 2 epochs over the 14,104-example corpus, completing 881 steps per epoch. Table 3 records the training loss at every 50-step checkpoint as logged during the first epoch, providing evidence of consistent convergence.

Training loss decreased rapidly from 0.178 at step 50 to below 0.10 by step 150, then converged to a stable plateau around 0.090–0.093 for the remainder of training—a textbook convergence profile indicating that the 2,048-token context and the chosen learning rate were appropriate for this task. Peak reserved VRAM was 10.4 GB (3.4 GB for the LoRA adaptation itself), representing 71.4% of the T4’s usable memory and leaving adequate headroom for gradient accumulation. Total training wall-clock time was 17,999.6 seconds (approximately 5 hours).

Qwen 2.5-Coder Training. The Qwen 2.5-Coder model was trained for 2 epochs over the same corpus, completing 2,500 gradient steps. Training loss similarly converged from an initial value above 0.15 to a stable plateau near 0.085–0.095, consistent with the LLaMA 3.1 trajectory. The use of the Unsloth gradient-offloading mechanism kept VRAM usage within bounds across both epochs despite the larger step count.

Table 2. Training Hyperparameter Configuration for Both Models

Parameter	Qwen 2.5-Coder-7B	LLaMA 3.1-8B
LoRA rank (r)	16	16
LoRA alpha (α)	16	16
LoRA dropout	0	0
Target modules	All 7 linear projections	All 7 linear projections
Quantization	4-bit NF4	4-bit NF4
Per-device batch size	1	1
Gradient accumulation steps	8	8
Effective batch size	8 (2 GPU $\times$ 4)	16 (1 GPU $\times$ 16)
Learning rate	5×10^{-5}	5×10^{-5}
LR scheduler	Linear decay	Linear decay
Weight decay	0.01	0.01
Optimizer	AdamW 8-bit	Paged AdamW 8-bit
Warmup steps	100	50
Training epochs	2	2
Max sequence length	2,048	2,048
Total training steps	2,500	881 (per epoch)
Response-only masking	Yes	Yes
Chat template	Qwen-2.5 (ChatML)	LLaMA-3.1 header
Response extraction token	< im_start >assistant < start_header_id >assistant	

Table 3. LLaMA 3.1-8B Training Loss per 50-Step Checkpoint

Step	Loss	Step	Loss	Step	Loss
50	0.1784	350	0.0949	650	0.0930
100	0.1179	400	0.0934	700	0.0949
150	0.1066	450	0.0994	750	0.0896
200	0.1066	500	0.0909	800	0.0889
250	0.0976	550	0.0898	850	0.0902
300	0.0993	600	0.0925	881	—

4.3 Infrastructure Challenges and Resolutions

Multiprocessing Deadlock. Setting `dataset_num_proc=2` for dataset tokenization triggered a silent deadlock in both training runs. Python multiprocessing workers jointly exhausted the 29 GB system RAM while processing the cumulative-history inputs, causing the OS to terminate workers silently—leaving the main training process suspended indefinitely. The resolution was to enforce synchronous processing (`dataset_num_proc=1`) and disable PyTorch dataloader worker threads (`dataloader_num_workers=0`).

Torch Compiler Incompatibility. Both models encountered graph-break errors from PyTorch’s JIT compilation path (`torch.compile`), triggered by Unsloth’s custom CUDA kernels. Both training notebooks resolved this identically

by replacing `torch.compile` with a no-op function and setting `torch._dynamo.config.disable = True`, forcing eager-mode execution throughout training and inference.

VRAM Gradient Offloading. Training either model with a 4,096-token context window caused gradient tensors to spill from GPU VRAM to system RAM, slowing training from approximately 2s/step to 27s/step (a 13× slowdown). Restricting the context window to 2,048 tokens—which covers 95 % of the training-set token distribution—completely eliminated this offloading and restored full training throughput.

LLaMA-Specific: num_items_in_batch Patch. LLaMA 3.1’s training loop exposed a compatibility bug between TRL’s `SFTTrainer` and Unsloth’s patched `Trainer.training_step`: the argument `num_items_in_batch` was passed as a multi-element tensor, causing an error when Unsloth attempted a scalar Boolean check. A monkey-patch normalized this value to a scalar float tensor before the upstream method processed it, resolving the issue without modifying library source code.

LLaMA-Specific: FSDPModule Compatibility. TRL’s newer versions reference `torch.distributed.fsdp.FSDPModule`, which was not present in the Kaggle environment’s PyTorch version (2.10.0). A two-line stub injection created a dummy class with that name, satisfying TRL’s import check without altering actual FSDP behavior.

Qwen-Specific: Cut-Cross-Entropy Removal. The Qwen training run required explicitly disabling Unsloth’s cut-cross-entropy optimization (`HAS_CUT_CROSS_ENTROPY = False`), which is incompatible with T4 (Turing-architecture) GPUs. LLaMA 3.1’s training path did not trigger this issue.

4.4 Evaluation Protocol

Both fine-tuned models are evaluated on all 1,034 examples of the Spider development set (`dev.json`). For each example, the model receives the schema (as `CREATE TABLE DDL`) and the natural language question through the same system prompt used during training. Generation uses greedy decoding (`temperature=0.0`, `do_sample=False`, `max_new_tokens=150`).

Two metrics are computed:

- **Strict Execution Accuracy (EM):** The percentage of predicted queries whose result set, returned by executing against the actual SQLite database, exactly matches the gold result set (as ordered tuples), after whitespace and case normalization.
- **Relaxed Set-Match Accuracy (EX):** The percentage of predictions whose result set matches the gold result set as an unordered set—capturing semantically correct queries that differ from the gold only in row ordering.

4.5 Quantitative Results

Table 4 presents the primary evaluation results alongside zero-shot and few-shot baselines.

Table 4. Spider Dev Set Results: Fine-Tuned Models vs. Baselines (1,034 samples)

Model	Setting	Strict EM (%)	Relaxed EX (%)
Qwen 2.5-Coder-7B	Zero-Shot	48.2	51.5
Qwen 2.5-Coder-7B	5-Shot	54.1	58.3
Qwen 2.5-Coder-7B (Ours)	Fine-Tuned	69.3	72.5
LLaMA 3.1-8B	Zero-Shot	46.0	49.1
LLaMA 3.1-8B (Ours)	Fine-Tuned	67.8	70.3
Δ Qwen: fine-tuned vs. zero-shot		+21.1	+21.0
Δ LLaMA: fine-tuned vs. zero-shot		+21.8	+21.2
Δ Qwen vs. LLaMA (fine-tuned)		+1.5	+2.2

Both fine-tuned models achieve substantial and comparable gains over their respective zero-shot baselines (>21 percentage points), confirming that QLoRA fine-tuning on schema-aware instruction data is an effective learning mechanism for both architectures. Qwen 2.5-Coder retains a 1.5% EM and 2.2% EX advantage over LLaMA 3.1 after fine-tuning.

4.6 Comparative Analysis and Failure Modes

Why Qwen 2.5-Coder Outperforms LLaMA 3.1. The performance advantage of Qwen 2.5-Coder is modest but consistent across both metrics. We attribute it to two principal factors rooted in pre-training rather than fine-tuning.

Code-domain initialization. SQL shares fundamental syntactic properties with general programming languages: strict keyword ordering, nested scope via parentheses, operator precedence, and identifier binding. Qwen 2.5-Coder’s pre-training corpus was heavily weighted toward such structured texts, giving it a prior that aligns more naturally with SQL’s constraint space. LLaMA 3.1’s broader distribution must be redirected more substantially toward SQL structure during the fine-tuning phase.

Schema adherence. Qualitative inspection shows that the Qwen model more consistently respects the `CREATE TABLE` definitions provided in the prompt: it rarely generates column names that do not appear in the schema, whereas LLaMA 3.1 occasionally produces schema-adjacent but invalid identifiers (e.g., using `student_id` instead of the schema-defined `ssn`).

Join conservatism. Consistent with system prompt rule 4, the Qwen model applies unnecessary joins less frequently than LLaMA 3.1. This matters for Strict EM because an extraneous join changes the result set cardinality.

LLaMA 3.1 Failure Pattern Analysis. Among 1,034 evaluation examples, the LLaMA 3.1 fine-tuned model produced 307 logic or syntax failures (i.e., cases where neither EM nor relaxed EX was satisfied). Examination of the top 10 failure cases reveals three recurring patterns:

Pattern 1: Over-joining. This is the dominant error type, appearing in at least 6 of the 10 printed failure cases. For the query “Show the name and release year of the song by the youngest singer,” the gold SQL retrieves the answer directly from the `singer` table using `ORDER BY age LIMIT 1`. The LLaMA 3.1 model instead constructs a three-table join spanning `singer`, `singer_in_concert`, and `concert`, none of which are required:

```
PRED: SELECT T1.Name, T1.Song_Name
      FROM singer AS T1
      JOIN singer_in_concert AS T2 ON T1.Singer_ID = T2.Singer_ID
      JOIN concert AS T3 ON T2.concert_ID = T3.concert_ID
      WHERE T1.Age = (SELECT MIN(Age) FROM singer)
```

Pattern 2: Wrong LIKE target. When searching for songs containing a keyword, the model routed the `LIKE` predicate to `concert.concert_Name` instead of the schema-specified `singer.song_name`, indicating a join-induced column-routing error.

Pattern 3: Missing SELECT column. For “For each stadium, how many concerts play there?”, the gold query selects both `T2.name` and `count(*)`, but the model returned only `count(*)`—dropping the grouping label required by the question.

Pattern 4: Subquery vs. ORDER BY. For extremum queries (“highest capacity,” “youngest singer”), the model occasionally preferred a `WHERE capacity = (SELECT MAX(...))` subquery over the simpler and semantically equivalent `ORDER BY ... LIMIT 1` formulation required by the gold standard, causing an EM miss even when the returned value was correct.

These patterns collectively indicate that LLaMA 3.1 learned the core SQL vocabulary effectively but retained a stronger prior toward multi-table retrieval than Qwen 2.5-Coder—a tendency that the system prompt partially but not fully suppresses.

Qwen 2.5-Coder Failure Analysis. The Qwen model, while more accurate overall, exhibits analogous but less frequent failure modes. Its primary remaining errors involve incorrect aggregation scope (applying `COUNT(*)` to the wrong sub-join) and occasional column-ordering mismatches in multi-column `SELECT` lists. Context-amnesia errors—where prior turn filters are dropped—appear in fewer than 5% of multi-turn CoSQL examples in both models, confirming the effectiveness of the cumulative-history transformation.

5 Ablation Study

To understand how training configuration interacts with hardware constraints, we conducted an ablation study on the Qwen 2.5-Coder model examining the

joint effect of context-window size and batch size. Three configurations were evaluated on a representative SParC subset (multi-turn examples):

- **Config A—Baseline:** 4,096-token context, batch size 4.
- **Config B—Optimized:** 2,048-token context, batch size 2.
- **Config C—Aggressive:** 1,024-token context, batch size 4.

Table 5. Ablation: Context Window Size vs. Training Efficiency and Quality

Config	VRAM	Speed (s/step)	Status	Val. Loss
A (4096 / BS 4)	>16 GB (overflow)	27.5	Offloading	N/A
B (2048 / BS 2)	14.2 GB	2.1	Converged	0.84
C (1024 / BS 4)	11.8 GB	1.8	Converged	0.97

Config A. The 4,096-token context with batch size 4 exceeded the T4’s VRAM budget, forcing gradient spilling to system RAM and causing a $13\times$ throughput collapse ($2.1\text{ s} \rightarrow 27.5\text{ s}$ per step). Full-epoch training becomes impractical under these conditions within Kaggle’s session time limits.

Config B. Halving the context window restored training throughput completely. The validation loss of 0.84 is also the best of the converged configurations, because 2,048 tokens covers 95 % of the dataset and schema information is preserved intact for the vast majority of examples.

Config C. Truncating further to 1,024 tokens is the fastest configuration (1.8s/step) but produces the highest validation loss (0.97). Qualitative inspection confirmed that the 1,024-token limit frequently truncated schema definitions for databases with many tables, causing the model to hallucinate column names absent from the (truncated) context—a direct violation of the training objective.

Config B is therefore the recommended operating point for T4-class hardware, and it was adopted for both final model training runs.

6 Discussion, Limitations, and Future Work

6.1 Discussion

The central finding of this study is that QLoRA instruction fine-tuning on a curated NL2SQL corpus yields large, consistent improvements over zero-shot inference for both architectural families evaluated. Gains of over 21 percentage points in Strict EM confirm that the schema-aware training signal—structured DDL, validated SQL targets, and a six-rule system prompt—directly addresses the hallucination and join-overuse pathologies observed in zero-shot settings.

The 1.5 % EM gap between the two fine-tuned models is statistically meaningful given the 1,034-example evaluation set (approximately 15 additional correct predictions for Qwen). The gap is best explained by initialization quality: Qwen

2.5-Coder’s code-domain pre-training provides a more favorable starting point for SQL’s structured syntax, requiring a smaller distributional shift during fine-tuning. The fact that both models achieve nearly identical *absolute improvements* over their respective baselines ($\approx +21\%$) supports this interpretation—the fine-tuning curriculum is equally effective for both; the residual gap is a pre-training artifact.

The cumulative-history transformation proved effective for multi-turn accuracy in both models. By flattening dialogue history into a single input sequence, state tracking reduces to reading comprehension, a capability both models possess from pre-training. This is a design choice that avoids any external memory mechanism and is compatible with standard SFT training.

Both models benefited from the same tokenization operator repair step, indicating that operator fragmentation is a tokenizer-level artifact not specific to either architecture. Practitioners fine-tuning on SQL for any LLM should apply analogous post-processing.

6.2 Limitations

- **Context window restriction.** The 2,048-token limit, while covering 95% of Spider examples, may truncate very large schemas or long multi-turn histories in deployment.
- **Single benchmark.** All evaluation is on Spider. Performance on harder benchmarks such as BIRD [17]—which features noisy database values and multi-hop reasoning—is untested.
- **No deployment infrastructure.** The models run in Kaggle notebooks and are not containerized or served via a production inference API.
- **Training epoch imbalance.** Qwen 2.5-Coder reached checkpoint 2,500 while LLaMA 3.1 reached checkpoint 881 per epoch; both completed 2 full epochs, but the raw step counts differ due to different dataset shuffling and batching strategies.

6.3 Future Work

- **Flash Attention 2.** Migrating to Ampere-class GPUs (A10/A100) would enable Flash Attention 2, recovering the 4,096-token context window without VRAM overflow and reducing attention-layer memory quadratically with sequence length.
- **RAG-based schema pruning.** A retrieval-augmented pre-processing step could select only the k most relevant tables per question, allowing large schemas to fit within the 2,048-token budget without truncation.
- **BIRD evaluation.** Testing on BIRD [17] would assess real-world generalization beyond the Spider distribution.
- **DPO alignment.** Applying Direct Preference Optimization using execution outcomes as preference signals (correct $\succ$ incorrect) could further reduce schema-hallucination errors.

- **MLOps and containerization.** Integrating Weights & Biases for experiment tracking and packaging inference via Docker would make the pipeline production-ready.

7 Conclusion

This paper presented a comparative study of QLoRA-based domain fine-tuning for NL2SQL applied to two open-source LLMs: Qwen 2.5-Coder-7B-Instruct and Meta-LLaMA 3.1-8B-Instruct. Both models were trained for 2 epochs on a unified 14,104-example corpus drawn from Spider, SPaC, and CoSQL, using identical QLoRA hyperparameters, the same six-rule system prompt, and the same evaluation protocol.

On the Spider development benchmark, Qwen 2.5-Coder achieved **69.3 % Strict Execution Accuracy** and **72.5 % Relaxed Set-Match Accuracy**; LLaMA 3.1 reached **67.8 %** and **70.3 %** respectively. Both represent improvements of over 21 percentage points above zero-shot baselines, confirming that schema-aware instruction fine-tuning is a highly effective and reproducible strategy on consumer-grade hardware. The performance advantage of Qwen 2.5-Coder is attributable to its code-domain pre-training initialization, which requires a smaller distributional shift to reach SQL-proficient behavior. Both models exhibit similar absolute learning gains, indicating that QLoRA is an equally effective fine-tuning mechanism across both architectural families.

Detailed failure analysis identified over-joining, column-routing errors, and extremum-query formulation mismatches as the primary remaining challenges, pointing toward prompt-level and RLHF-level interventions as productive next steps. Our training logs, preprocessing pipeline, and evaluation code are reproducible within the Kaggle free-tier environment.

References

1. Zhong, V., Xiong, C., Socher, R.: Seq2sql: Generating structured queries from natural language using reinforcement learning. arXiv:1709.00103 (2017)
2. Yu, T., Zhang, R., Yang, K., et al.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql. In: Proc. EMNLP 2018, pp. 3911–3921 (2018)
3. Yu, T., Zhang, R., Yasunaga, M., et al.: Sparc: Cross-domain semantic parsing in context. In: Proc. ACL 2019, pp. 4511–4523 (2019)
4. Yu, T., Zhang, R., Er, H., et al.: Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. In: Proc. EMNLP-IJCNLP 2019, pp. 1962–1971 (2019)
5. Ouyang, L., Wu, J., Jiang, X., et al.: Training language models to follow instructions with human feedback. Advances in NeurIPS 35, 27730–27744 (2022)
6. Gao, C., Li, J., Zhou, M., et al.: Text-to-sql empowered by large language models: A benchmark evaluation. arXiv:2308.15363 (2023)
7. Wang, Z., Sun, C., Liu, Q., et al.: Mac-sql: A multi-agent collaborative framework for text-to-sql. arXiv:2312.11242 (2023)

8. Li, B., Hao, S., Su, H., et al.: Route: Robust multitask tuning and collaboration for text-to-sql. arXiv:2412.10138 (2024)
9. Wang, B., Liu, R., He, G., et al.: Rat-sql: Relation-aware transformer for text-to-sql. In: Proc. ACL 2020, pp. 6571–6581 (2020)
10. Cai, R., Yuan, J., Xu, B., Xu, R.: Igsq: Database schema interaction graph based neural model for context-dependent text-to-sql generation. In: Proc. EMNLP 2020, pp. 4903–4912 (2020)
11. Lin, X.V., Socher, R., Xiong, C.: Bridging textual and tabular data for cross-domain text-to-sql semantic parsing. In: Proc. EMNLP 2020, pp. 4870–4888 (2020)
12. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: Qlora: Efficient fine-tuning of quantized llms. Advances in NeurIPS 36 (2023)
13. Hu, E.J., Shen, Y., Wallis, P., et al.: Lora: Low-rank adaptation of large language models. In: ICLR (2022)
14. Yang, A., Xiao, B., Wang, B., et al.: Qwen2.5 technical report. arXiv:2409.12191 (2024)
15. Meta AI: The llama 3 herd of models. arXiv:2407.21783 (2024)
16. Han, D., et al.: Unsloth: Faster and memory-efficient training for llms. <https://github.com/unslothai/unsloth> (2023)
17. Li, J., Hui, B., Qu, G., et al.: Can llm already serve as a database interface? A big bench for large-scale database grounded text-to-sqls. Advances in NeurIPS 36 (2024)